Appendix J — OpenScript

The bytecode, the keyword pool, the handlers that have actually been expanded, and the subset a clean-room player implements. Chapters 6 and 14 are the claim list. This appendix is the working description.

Date: 16 October 2026

OpenScript in an 8.1 book is not stored as the text you type in Instructor. The handler name is often plain text. The body is a compressed token stream. `TB80CMP` compiles and expands that stream. `TB80BAS` evaluates expressions. Neither module contains the keyword pool as a C string table. A search of BAS for `buttonClick` finds nothing. That is not evidence the scripts are missing.

J.1 Who owns it

Piece	Where	What it is
Keyword pool	`TB80CMP` NE resource `0x77`, type 100, id 119	The names the expander prints. 15776 bytes.
Second emit routine	Resource `0x78`, id 120	15616 bytes. An early dump counted 980 strings. It is not a second language.
Property names	`COMPILERPROPINFOFROMINDEX` locks resource `0x70` (id 112, 3872 bytes, 213 strings). The ids the books use are also BAS resource type `0x2D1`, id `0x2D0`.	`bounds` is `0x4004`. Not the keyword pool.
Expand	`FUN_1040_0970` in CMP. The rename list calls it `ExpandTextCore`.	The byte classes below are this function.
Compile	`COMPILERCOMPILETEXT`, CMP ordinal 1	Not used by a lesson open.
Expand export	`COMPILEREXPANDTEXT`, CMP ordinal 3	The stable breakpoint if a compile test is run.
Execute / evaluate	`CFUNEXECUTE` ordinal 8, `CFUNEVALUATE` ordinal 9	
Arithmetic	`FUN_1008_2d31` in BAS, reached from the switch at `1008:2DC7`	148 cases. FPU: `FLD`, `FSTP`, `FCLEX`. Not the record reader.
Small runtime words	`TB80RTM` file `0xAACAD`	`handle`, `get`, `set`, `notifybefore`, `notifyafter`, `true`, `false`, `to`. Not the compiler table.

Do not treat every locked resource near `0x70` as the keyword list. `0x70` is properties. `0x77` is keywords.

An early string dump of `0x77` counted 992 strings. The decoder that produced the 76 Logic Families handlers counts 994 keywords. Use 994. The offset table starts at byte 2 plus count times 4. A loader that used count times 2 was wrong before the back-reference bug was even found. The 994 names have not been locked against a script compiled in Instructor 8. A name in that table is a string in the resource. It is not, by itself, proof that NIDA emits it.

J.2 The byte classes

From `FUN_1040_0970`. A byte is classified once. The numeric ranges overlap, so a second pass that says “everything above `0xBF` is a back-reference” will also eat the whitespace opcodes.

Byte	Meaning
`0x00`–`0x7F`	Literal character. This is how a quoted page id survives.
`0x80`–`0xBF`	Keyword.
`0xC0`–`0xDF`	Back-reference, after the whitespace classes have been taken out. Copy from the compressed input, not from the output.
`0xE0`–`0xEF`	CR, LF, and spaces.
`0xF0`–`0xFE`	Spaces.
`0xFF`	Repeat.

Keyword index, inside `0x80`–`0xBF`:

```
index = byte & 0xF
```

```
if bit 4 is set:
    index = (index << 8) | next_byte
if bit 5 is clear:
    append a space after the keyword
```

Bit 4 and bit 5 are bits of the opcode, not of the keyword text. A space is part of the encoding. It is not a separate literal unless the script had a space the keyword rule did not already emit.

Back-reference, as fixed on 10 October 2026:

```
if bit 4 is clear:
    length = (byte & 0xF) + 3
    src = (pos + 1) + (-4 - next_byte)
else:
    length = (byte & 0xF) + 4
    src = (pos + 1) + (-1 - word_offset)
copy length bytes from input[src]
```

`pos` is the position of the opcode in the compressed buffer. `src` can fall before the slice you passed the decoder. On `tb80r.sbk`, reading those from the whole book dropped the `?` marks from 14256 to 66. The 66 that remain are question marks in the script, not failed copies. A windowed decoder will mark a real back-reference as garbage. That is the decoder's window, not a different encoding.

J.3 Two bugs that produced the wrong language

`tbworkingpapers/tbwp0001/openscript_notes.md` begins by saying the decoder emits `italic`, `third`, `fourth`, and `systime`, and that the bytes might not be OpenScript. That opening is superseded by the same file's later sections. Do not cite it.

Two independent mistakes:

1. The keyword offset table was read as count times 2. Keywords came from the wrong row. `systime` appeared where the script says something else.
2. Bytes `0xC0`–`0xFF` were copied from the output stream. That produced `keySlash` and truncated words.

After both fixes the same bytes are:

```
system lastbook
  lastbook = name of this book
  system nextpage
  nextpage = name of this page
  go to page "P20001" of book "nida81.tbk"
end
```

Other handlers that are real source, not Latin-1 of the bytecode: `send next`, `send previous`, `go to page "P400"`, `go to page "P310" of book "nida81.tbk"`.

Keywords live in `TB80CMP`. BAS has no `handle` or `buttonClick` strings. RTM's eight words at `0xAACAD` are not a substitute table. There is no separate NIDA keyword resource in `nc81int.dll` or `nc81rec.dll`. Those DLLs are trainer and session calls.

J.4 How a handler sits in the file

A lesson book stores the name as plain text and compresses the body.

```
To Handle EnterPage
<compressed body>
```

The capitalization is not stable. Logic Families has 46 handlers whose text is `TO HANDLE buttonClick`, 29 whose text is `To Handle ButtonClick`, and 1 `To Handle EnterPage`. That is the 76. An extractor that required a plain `END` found 22 of them. 76 is the count after the input-stream fix. The corpus tool stops at the first standalone `end` and drops what follows. That is the tool. It is not a claim that `COMPILEREXPANDTEXT` itself stops there.

`tb80r.sbk` compresses the prefix too. There is almost no plain `to handle` in the file. A scan that decodes windows and keeps those whose output starts with `to handle` or `to get` recovered 33 handlers. The encoding rules match the lesson books. The histogram does not: more `0xFF` and `0xE0`, fewer `0xBF` and `0xC0`-range copies. An early pass that only searched for the plain prefix reported zero handlers in `tb80r.sbk`. That pass is superseded for this one file.

Book	Handlers	Storage
`9e954d01.tbk`	76, on 39 of 129 pages	Plain prefix, compressed body.
`nida81.tbk`	100	All `buttonClick`. Same encoding. The shell is a book.
`tb80r.sbk`	33	Prefix compressed. Reader book: help, bookmarking, logging, path manager, popup page manager. Not the engine.
`nc81base.sbk`, `nc81recs.sbk`, `nc81test.sbk`, `nc81trnr.sbk`	0 decoded	High-byte runs do not decode as this bytecode. Early pattern counts (61, 17, 94, 62) were not handlers.
`tb80anm.sbk`, `tb80hyp.sbk`	0 decoded	Strings and menus.

“No script in any system book” is too strong for `tb80r.sbk` and still true, as decoded text, for the four `nc81*` books. Those four are resource containers: the object-type name table and menus. The shell behavior that looks like a system book is `nida81.tbk` plus `nc81int.dll` and `nc81rec.dll`.

A `linkDLL` is not a system book. `TBDLG`, `TBWIN`, `TBDOS`, `TBREG`, `TBDB3`, `TBPDX`, and `TBCBT` add commands only when a handler links them. `TBWIN` is `INIT_TBKWIN` and `TBM_EVALUATE`. None of those walks page records.

J.5 What the decoded handlers do

Logic Families is a menu of jumps. It is not a self-running program. Navigation is not “a quoted page id and nothing else.” P20000 of `nida81.tbk` goes to P20001 because the `buttonClick` handler says so.

Page	What was read
P900	Lesson menu. TTL → P1001. Other → P2001. Final quiz → P10001. Exit → `nida81` P20001. No Next.
P1001, P2001	Back returns to P900, not to `nida81` P20001.
P300	`selection1` is the IBM PC Compatible Computer line. `buttonClick` sends `lastbook nida81.tbk` and `nextpage` P310.
P10001	Assessment. Directions → `nida81` P10020. Exit → `nida81` P20001. Begin → P10210.
P10410	Section end. Menu returns to P900.
P1290	Quiz. Correct choice 5408. Again → P1292. Next → P1300.

A retry page returns to the quiz whose `again` field names it. `initializeRecs` on the shell clears responses when a session loads. That sentence is inferred from the shell script. It has not been watched as a call into `nc81rec.dll`.

The words the courses actually use:

`go to`, `send`, `send next`, `send previous`, assignment, `put into`, `set to`, `get`, increment and decrement, nested `if` / `else` / `end if`, `step from` / `to`, `name of this book`, `name of this page`, `system`, and a quoted page or book name.

One decoded `enterPage` starts `system logical __import`, sets `lastbook` to `name of this book`, sets `nextpage` to `"p10"`, and branches on `__import`. Another button is a comment `--{Go to next page}` and then `send next`.

`9e954d01.tbk` at file `0xB1A96` names `tb80r.sbk`, `nc81trnr.sbk`, `nc81base.sbk`, `nc81recs.sbk`, and `nc81test.sbk`, and the script calls `initializebase`, `initializerecs`, `initializetest`, `openSession`, and `gotoPageList`. It also jumps to `nida81.tbk` and to `93954302.tbk`. Those initialize names have not been watched as

a call log. The order in `toolbook.lst` has not been watched live.

A TBX `.script` file is the same handler as plain text. It is a playback path for the export. It is not a reader of the `.tbk`. The export of this lesson has 302 `.script` files. The binary has 76 decoded handlers in the lesson plus 100 in the shell. Those counts are not required to match. The export splits what the binary stored as one compressed body, and it includes pages the 8.1 file does not have (P440 and P2920).

J.6 Expressions

`FUN_1008_2d31` stays in BAS segment `1008`. It looks for names such as `log`, then uses the FPU. The clean-room evaluator copies the operations that switch reaches:

`+`, `-`, `*`, `/`, `abs`, `sqrt`, `log`, `exp`, `sin`, `cos`, `atan`, `asin`, `acos`.

Anything else is left as the original text. The function is not the record reader and not a paint path. An earlier engine treated the right-hand side of an assignment as an opaque string. The named operations above are the part that changed.

J.7 What the clean-room player does and does not do

Accepts, as of the player that ships with this record:

- `send NAME to TARGET`
- a `get` that is a call
- `forward to TARGET`
- `break NAME`
- `PROP of TARGET`
- the navigation and assignment subset in J.5
- the arithmetic in J.6

Records, and does not evaluate: predicates such as `is not in`.

Ignores: `local` and `system` declarations. They decode. They do not allocate.

Does not apply: property writes, to a live object. Setting `bounds of` something does not move a widget.

Not implemented, or only sketched: nested property expansion, opcodes past the common subset, and `forward` into a handler that lives in a system book.

The CMP expand host that died in `TB80W16` at `001c:00000015` (`9A 9C 06 57 2A`, segment not present) is parked. It is not the NIDA open path. Do not single-step that far call to learn a keyword.

J.8 The compile that would lock the table

Instructor 8 can compile OpenScript. It cannot write TBX. TBX starts in 9.5. The way to get bytecode out of 8.1 is to save a book and expand the handler.

1 On the Win98 or WinXP VM, open Instructor 8. Make a new book. Add a button. Paste:

```
to handle buttonClick
  go to page "P100"
end buttonClick
```

1 Save as `compile-test.tbk`. Copy it off the VM.

2 Expand with the decoder that already finds `to handle` / `end`, or break `COMPILEREXPANDTEXT` (CMP ordinal 3) if the expand is done inside Instructor.

3 A second script with `if` / `else` / `send` / `step` covers the rest of the common subset. A token that does not resolve is a gap in the 994-row table.

That test is written down. It has not been run. It does not trace the author UI. The author binaries (`TB80EDT`, `TB80.EXE`) are not in the archive.

J.9 What this appendix will not pretend

- The 994 strings are not printed here. They have not been sifted against a known compile. Printing them would repeat the symbol-map mistake.
- A high-byte run in an `nc81*` book is not a handler until it decodes to `to handle` or `to get`.
- `CDBCHUNKFIND` and the comma / CR / LF walker are text chunkers. They are not this expander.
- Ordinal 51 is not involved. A lesson open does not call it, and OpenScript does not create the object the script is attached to.